

연구 트랙

Team 14 (def)

팀 지도교수 : 심재형 교수님

SynTree KV

코딩 에이전트를 위한 AST 기반 KV 재사용 추론 최적화

2376142 신은서

2371046 이재린

2466071 서혜원

2026. 6. 21.

SynTree KV

코딩 에이전트를 위한 AST 기반 KV 재사용 추론 최적화

1. 서론

1.1. 연구 배경 및 문제 포착

에이전트 AI 는 다양한 도메인에 걸쳐 활용되고 있으며, 그 중에서도 코딩 에이전트는 현재 개발자들 사이에서 가장 활발히 도입되고 있는 유형 중 하나이다. 코딩 에이전트는 파일 읽기, 코드 수정, 테스트 실행을 수십 번 반복하는 멀티턴 루프로 동작한다. LLM 서빙 과정에서는 입력 토큰을 처리할 때마다 KV cache(Key-Value cache, 어텐션 연산 과정에서 생성되어 이후 토큰 계산에 재사용되는 중간 결과)가 생성되는데, 동일한 입력이 반복되면 이를 다시 계산하지 않고 재사용함으로써 연산량을 줄일 수 있다. 본 연구는 이러한 반복적인 파일 I/O 패턴이 규칙적으로 재등장하기 때문에 KV cache 재사용 효과를 극대화할 수 있는 도메인이라 판단하여 코딩 에이전트를 연구 대상으로 선택하였다.

실제 수집한 48 개 trace 분석 결과, 코딩 에이전트가 소비하는 전체 토큰의 83.3%가 파일 읽기(observation) 작업에서 발생함을 확인하였다. 특히 이 파일 읽기 결과의 상당 부분은 세션 전반에 걸쳐 반복적으로 등장하는 동일한 코드 블록으로 구성되어 있었다. 그러나 vLLM 과 같은 Serving Layer 는 이러한 중복을 인지하지 못한 채 매 턴마다 컨텍스트 전체를 처음부터 prefill 하며, 이는 처리한 정보를 매 턴 다시 계산하는 불필요한 비용 낭비로 이어진다.

1.2. 문제의 정의

기존 vLLM 의 prefix caching 은 앞부분 토큰이 완전히 동일할 때만 KV cache 를 재사용한다. 그러나 멀티턴 세션에서는 타임스탬프, 도구 호출 ID 처럼 매 턴 바뀌는 동적 메타정보가 끼어들어 재사용 조건이 깨진다.

실험 결과, 1,721 턴에 걸쳐 누적 40 만 건 이상의 prefix cache 블록 hit 가 발생했음에도 TTFT(Time-To-First-Token, 사용자 요청 후 첫 번째 출력 토큰이 생성되기까지 걸리는 시간) 개선은 2.6%에 그쳤다. 이는 prefix 가 일치하는 구간까지는 재사용이 이루어지지만, 동적 메타정보로 재사용 조건이 깨지는 시점부터는 그 이후 전체가 다시 처음부터 prefill 되기 때문이다. 이에 본 연구는 "코딩 에이전트의 멀티턴 세션에서 중복 코드 블록이 반복 등장함에도, 토큰 단위 완전 일치에 의존하는 기존 caching 방식은 이를 인식하지 못해 불필요한 지연 비용이 누적된다"는 문제를 정의하였다.

1.3. 대상 및 기대 효과

본 연구 결과의 직접적인 활용 주체는 코딩 에이전트를 자체 인프라에서 서빙하는 개발팀, 즉 OpenHands 와 같은 에이전트 프레임워크를 vLLM 등의 Serving Layer 위에서 운용하는 조직이다. 멀티턴 세션이 누적될수록 prefill 비용이 함께 증가해 응답 지연과 GPU 자원 소모가 비례해서 커지는 구조이므로, 본 연구가 제안하는 KV cache 재사용 방식이 적용되면 추가 하드웨어 투자 없이 동일한 Serving Layer 위에서 TTFT 를 개선하고 GPU 자원 활용 효율을 높일 수 있을 것으로 기대된다.

1.4. 솔루션 방향 및 차별성

이러한 기대 효과를 실현하기 위해, 본 연구는 1.2 에서 정의한 문제에 대한 해결책을 모색하였다. 초기에는 자주 읽히는 코드 블록의 KV cache 를 메모리에 고정(pin)하는 방식으로 이를 해결하고자 하였다. 그러나 KV cache 에는 위치 정보(RoPE, Rotary Position Embedding — 토큰의 순서를 attention 연산에 인코딩하는 방식)가 이미 인코딩 되어있어, 단순 고정만으로는 위치가 바뀌는 순간 재사용이 불가능하다는 근본적인 한계에 부딪혔다. 또한 빈도 기반 pinning 은 변수명만 다른 구조적으로 동일한 코드를 같은 블록으로 인식하지 못한다는 문제도 확인되었다. 이 두 가지 한계는 결국 재사용 판단의 기준이 토큰이 아니라 코드의 문법적 구조 자체여야 함을 시사한다.

기존 KV cache 재사용 연구는 대부분 토큰 단위(token-wise)로 재사용 대상을 결정하는 방식에 집중되어 있으며, 코드의 문법 구조를 단위로 삼는 AST 기반 선행 연구는 거의 존재하지 않는다. 이에 본 연구는 AST 서브트리 단위 KV cache 재사용 방식인 SynTree KV 를 채택하였다.

2. 선행 연구

2.1. 기존 연구 접근 – Related Works

기존 접근은 크게 세 갈래로 나뉜다.

2.1.1. 정확한 prefix 재사용 접근 – PagedAttention (vLLM)

vLLM(Virtual Large Language Model—오픈소스 LLM 추론 서빙 엔진)의 PagedAttention 은 연속된 앞부분 토큰이 완전히 일치할 때만 KV-cache 를 재사용하는 산업 표준 방식이다. 구현이 간단하고 정확도 손실이 없으나, 타임스탬프·도구 호출 ID 처럼 턴마다 바뀌는 메타정보가 prefix 에 끼어들면 hit rate 가 0 에 가깝게 떨어진다.

2.1.2. 모듈형 KV 재사용 접근 – PromptCache, CacheBlend

PromptCache 는 사전에 정의한 스키마 단위로 KV-cache 를 재사용하고, CacheBlend 는 비-prefix 구간의 KV 를 선택적으로 재계산하여 품질을 보정하는 방식이다. 두 기법 모두 정적으로 정의된 청크(Chunk — 자연어를 특정 크기로 나눈 단위) 경계를 전제하는데, 코딩 에이전트의 동적 도구 출력(파일 내용, grep 결과 등)에는 경계가 사전에 알려져 있지 않아 hit rate(LLM 이 텍스트를 생성할 때, 이전 대화나 프롬프트에서 이미 계산해 둔 데이터(Key 와 Value)를 다시 재사용하는 비율)가 낮다.

2.1.3. KV 압축·증류 접근 – SnapKV

SnapKV 는 어텐션 스코어를 기반으로 중요도가 낮은 KV 를 버리는 SOTA(State-of-the-Art — 최고 수준의 기술) 방법이다. 코드 구조 정보를 직접 활용하지 않고 KV 를 버리는 방식으로, 손실로 인한 정확도 하락 문제가 존재한다.

2.2. 기존 연구의 한계

기존 연구들을 검토한 결과, 다음 세 축의 교차점을 동시에 다룬 연구는 발견되지 않았다.

2.2.1. 멀티턴 코딩 세션 내 동적 경계 문제

vLLM prefix caching 은 동적 경계 문제를 해결하지 못한다. — 턴마다 변하는 메타정보가 prefix 를 깨뜨리는 순간 재사용이 불가능 해진다.

2.2.2. 코드 고유의 구조적 반복성 활용

PromptCache·CacheBlend 는 코드 고유의 구조적 반복성을 활용하지 못한다. — 정적 청크 경계를 전제하기 때문에 동적 도구 출력에는 적용이 어렵다.

2.2.3. 세션 수준 KV 재사용

SnapKV 류는 코드 구조 정보를 직접 활용하지 않는다 — 어텐션 스코어 기반 압축은 도메인 특수 지식을 시스템 최적화에 반영하지 않는다.

이에 본 연구는 AST 서브트리를 자연스러운 동적 재사용 단위로 삼고, 변수명 무관 지문(α -rename 과 BLAKE3)과 RoPE re-positioning 을 결합하여 이 세 축의 교차점을 공략한다.

2.3. 핵심 아이디어 및 기존 연구 대비 독창성

2.3.1. 동적 경계 처리 대신 AST 서브트리 단위 KV 재사용 적용

기존 연구 vLLM prefix caching, PromptCache, CacheBlend에는 모두 재사용 단위의 경계가 사전에 고정되어 있어야 한다는 전제를 공유한다. 그러나 코딩 에이전트의 동적 도구 출력(파일 내용, grep 결과 등)에는 경계가 사전에 알려져 있지 않아 이 전제가 성립하지 않는다.

따라서 본 연구에서는 Tree-sitter 로 입력 코드를 증분 파싱하여 AST 를 생성하고, 아래 조건을 만족하는 서브트리를 재사용 단위로 동적으로 추출한다. 경계를 사전에 정의할 필요가 없다.

- 1) 노드 유형 $\in \{\text{FunctionDef}, \text{ClassDef}, \text{ImportStmt}, \text{CallExpr}, \dots\}$
- 2) AST 깊이 ≥ 2 , 토큰 수 ≥ 8

2.3.2. 변수명 변화 허용 대신 α -rename 정규화와 BLAKE3 지문 적용

기존 연구에서는 토큰 수준의 exact-match 를 전제하기 때문에 변수명이 하나라도 달라지면 구조적으로 동일한 코드 블록도 다른 것으로 인식한다. 이는 코딩 세션에서 자주 발생하는 리팩터링, 인자명 변경 등의 상황에서 재사용 기회를 놓치게 만든다.

따라서 본 연구에서는 α -rename 정규화로 식별자를 VAR_0, VAR_1 과 같은 형태로, 리터럴을 LIT 으로 치환한 뒤 S-expression 으로 직렬화하고 BLAKE3 해시로 32 바이트 지문을 생성한다. 변수명만 다를 뿐 구조가 동일한 코드는 동일한 지문으로 인식되어 단순 해시 캐시가 잡지 못하는 반복 패턴을 포착할 수 있다. 이 차별점은 α -rename 적용과 비적용 비교 실험에서 9.8%p hit rate 차이로 정량적으로 검증되었다.

2.3.3. RoPE 위치 불일치 문제 해결을 위한 RoPE re-positioning 적용

기존 연구 PromptCache 는 RoPE positional shift 를 처리하지 않아, 위치가 달라진 KV 를 그냥 splice 하면 어텐션이 왜곡되는 문제가 발생한다. 또한, CacheBlend 는 token-level positional re-encoding 을 수행하지만 서브트리 수준의 구조적 경계를 정의하지 않아 부분적인 해결에 그친다.

따라서 본 연구에서는 재사용 KV 를 삽입할 때 발생하는 위치 오프셋 Δ 를 $K_{new} = R(\Delta) \cdot K_{cached}$ 수식으로 완전히 보정한다. 이 단계는 PromptCache·단순 해시 캐시와의 알고리즘적 핵심 차별점이며, 제거 시 pass@1(pass@1 — 단 한 번의 시도로 정확한 정답을 생성해낼 확률(%))을 의미하는 평가지표)이 5pp 이상 추가 하락할 것으로 예측한다.

아래 표는 기존 연구와의 핵심 차별점을 요약한 것이다.

	vLLM Prefix	PromptCache	CacheBlend	SnapKV	본 연구
동적 경계 처리	X	X	X	–	O AST 서브트리
변수명 변화 허용	X	X	X	–	O α -rename
RoPE 위치 보정	X	X	Δ 부분적	–	O 완전 보상
정확도 유지	O	O	O	X 손실 발생	O

3. 시스템 아키텍처 및 기술적 솔루션

3.1. 실험 환경

본 연구의 실험은 다음과 같은 환경에서 수행되었다.

모델은 Qwen3-Coder-30B-A3B-Instruct-FP8 을 사용하였다. 이는 2026 년 5 월 21 일 이전 기준 OpenHands 프레임워크에서 공식으로 권장하는 코딩 에이전트 특화 모델이며, FP8 양자화(모델의 수치 표현 정밀도를 낮춰 메모리를 절약하는 기법으로, 성능 손실을 최소화하면서 대형 모델을 실용적인 환경에서 운용할 수 있게 한다)를 적용하여 메모리 사용을 효율화하였다.

하드웨어는 NVIDIA RTX 5090 2 장을 사용하였으며, `tensor-parallel-size 2` 로 두 GPU 에 모델을 분산하여 서빙하였다.

소프트웨어 스택은 다음과 같이 구성하였다. 서빙 프레임워크로는 vLLM 을 사용하였으며 `--enable-expert-parallel`, `--max-model-len 131072`, `--gpu-memory-utilization 0.9` 옵션을 적용하였다. 에이전트 프레임워크는 OpenHands 를 사용하였고, AST 파싱에는 증분 파싱을 지원하는 Tree-sitter 를, 지문 생성에는 BLAKE3 해시(16 자리)를 사용하였다.

벤치마크는 SWE-bench Lite 에서 25 개 이슈를 선정하였다. 총 12 개 레포지토리에서 비율에 맞게 랜덤으로 선택하였으며, 각 이슈를 2 회 실행하여 총 50 개의 trace 를 수집하였다.

3.2. 데이터 수집

코딩 에이전트가 실제로 작업을 수행하는 과정을 기록한 데이터(trace)를 수집하였다. 각 trace 에는 파일 읽기, 코드 수정, 터미널 명령 실행 등 에이전트의 전체 작업 흐름이 이벤트 단위로 기록된다.

수집된 50 개 trace 중 도구 호출을 명시하였지만 실제로 실행하지 않은 채 비정상 종료된 2 개(django_django-11019_run1, matplotlib_matplotlib-23563_run0)를 제외하고 최종 48 개를 분석에 사용하였다. 수집된 trace 의 평균 events 는 72.0 개, 최소 events 는 9 개, 최대 events 는 160 개였으며, ActionEvent 2 개 이하인 비정상 trace 와 ErrorEvent 로 종료된 trace 는 0 개였다.

기존 14B, 1.5B 모델 실험에서는 JSON 파싱 오류, 파일 경로에 대한 모델 hallucination 등으로 인해 50 개 중 정상 trace 가 소수에 그쳤으나, Qwen3-Coder-30B-A3B-FP8 모델로 전환한 이후 이러한 문제가 해소되었다. 또한 각 실험 시작 전 `git checkout . && git clean -fd` 를 추가하여 레포지토리의 상태를 초기화함으로써 trace 간 오염을 방지하였다.

trace 를 구성하는 이벤트는 크게 네 종류로 구분된다.

첫째, SystemPromptEvent(source : agent)는 세션 시작 시 모델에게 주어지는 시스템 프롬프트로, OpenHands 에이전트의 역할 정의, 사용 가능한 도구 목록, 응답 형식 지침을 담고 있다. 매 trace 당 정확히 1 개이며 항상 첫 번째 이벤트로 등장한다.

둘째, MessageEvent(source : user)는 사용자가 에이전트에게 내린 초기 작업 지시로, "이 repo 에서 이 이슈를 고쳐라"는 형태의 task 메시지이다. 매 trace 당 1 개씩 등장하며, 비정상 종료된 2 개 trace 에는 agent source 의 MessageEvent 도 추가로 존재한다.

셋째, ActionEvent(source: agent)는 에이전트가 결정한 행동인 tool 호출을 의미한다. tool_name 은 `file_editor`(command=`view` 인 경우 파일·디렉토리 읽기, command=`str_replace` 인 경우 코드 수정, command=`create` 인 경우 새 파일 생성), `terminal`(테스트·pip install 등 터미널 명령 실행), `think`(실제 도구 호출 없는 모델 내부 추론), `finish`(작업 완료 선언)로 구성되며, trace 당 평균 36 개가 등장하고 ObservationEvent 와 1:1 쌍을 이룬다.

넷째, ObservationEvent(source: environment)는 tool 실행 결과가 환경에서 돌아온 것으로, `file_editor view` 결과는 `cat -n` 형태의 파일 내용을, `terminal` 결과는 "Your thought has been logged."를, `finish` 결과는 완료 확인 메시지를 포함한다. AST hit rate 분석에서 실제 코드가 들어있는 핵심 이벤트가 바로 이 ObservationEvent 이다.

3.3. AST 기반 KV 재사용 파이프라인

본 실험의 핵심 아이디어는 코딩 에이전트의 멀티턴 작업 과정에서 반복적으로 등장하는 코드 구조를 AST(Abstract Syntax Tree, 추상 구문 트리) 단위로 식별하고, 이전에 계산해둔 KV 캐시를 재사용하여 불필요한 재계산을 줄이는 것이다.

전체 파이프라인은 다음 단계로 구성된다.

첫째, 텍스트 정제 단계에서는 `cat -n` 명령으로 삽입된 줄 번호 등 불필요한 요소를 제거하여 순수한 코드 텍스트만 남긴다.

둘째, Tree-sitter 를 사용하여 코드를 파싱하고 AST 를 생성한다. Tree-sitter 는 다양한 프로그래밍 언어를 지원하며 증분 파싱이 가능하여 실시간 처리에 적합하다.

셋째, 자격 조건 필터링 단계에서는 모든 AST 노드를 재사용 후보로 삼지 않고, 의미 있는 코드 구조만을 선별한다. 노드 타입이 `function_definition`, `class_definition`, `import_statement`, `import_from_statement`, `call`, `assignment` 중 하나에 해당하면서, AST 깊이가 2 이상이고, 토큰 수가 8 개 이상(Qwen3-Coder-30B tokenizer 기준)인 경우에만 재사용 후보가 된다.

넷째, α -rename 정규화 단계에서는 변수명이 달라도 구조가 동일한 코드를 같은 코드로 인식하기 위해 정규화를 적용한다. 구체적으로 식별자를 `VAR_0`, `VAR_1` 순으로 치환하고, 리터럴은 LIT 으로 통일한 뒤 S-expression 형태로 직렬화한다. 이 과정에서 모듈명·메서드명의 유지 여부에 따라 두 가지 방식을 비교 실험하였다.

마지막으로 BLAKE3 지문 생성 및 SessionCacheTable 조회 단계에서는 정규화된 S-expression 에 BLAKE3 해시 함수를 적용하여 16 자리 지문(fingerprint)을 생성한다. 이 지문으로 SessionCacheTable 을 조회하여 재사용 여부를 결정하며, hit 인 경우 이전에 계산해둔 KV 캐시를 불러와 재사용하고 위치가 달라진 경우 RoPE re-positioning 을 적용하여 위치 불일치를 보정한다. Miss 인 경우에는 정상적으로 prefill 을 수행한 뒤 결과를 테이블에 등록한다.

4. 성과

4.1. Hit rate 측정 실험

4.1.1. 실험 설계

파이프라인 구현 과정에서 다음 세 가지 설계 결정을 실험을 통해 검증하였다.

첫째, 노드 선정 및 토큰 카운트 기준이다. 전체 노드 방식과 최상위 노드 방식을 비교하였다. 최상위 노드 방식은 재사용 단위를 서브트리 하나로 명확히 정의하며, 상위 노드 재사용 시 하위 KV 가 자동으로 재사용되는 구조와 일치한다는 장점이 있다. 토큰 카운트의 단위는 실제 LLM 의 tokenizer(텍스트를 모델이 처리하는 최소 단위인 토큰으로 분할하는 도구, 기준 모델: Qwen3-Coder-30B)로 토큰 수를 세었다.

둘째, α -rename 방식이다. all 방식과 vars_only 방식(모듈명·메서드명 보존 방식)을 비교하였다. all 방식은 모든 식별자를 VAR_N 으로 치환하고 리터럴은 LIT 으로 정규화 한다. 반면 vars_only 방식은 변수·함수명만 VAR_N 으로 치환하고 모듈명·메서드명은 보존하는 방식이다. 이 방식은 의미가 다른 두 함수가 다른 지문(fingerprint)로 변환된다.

셋째, 분석 소스이다. ObservationEvent(파일 읽기 결과, 터미널 출력)만 분석한 경우와 ActionEvent 의 old_str, new_str 을 함께 포함한 경우를 비교하였다. observation 단독으로도 hit rate 32.6%로 GO 기준 25%를 여유 있게 통과하였으며, action 을 포함한 경우 8.3%p 상승하였다. action 이 수치를 인위적으로 과도하게 끌어올리지 않는다는 점에서, observation+action 방식도 유효한 분석 기준으로 채택 가능하다고 판단하였다.

4.1.2. 실험 결과

4 가지 조건(분석 소스 × α -rename 방식)으로 실험한 결과는 다음과 같다.

조 건	hitrate	token ratio	eligible	hits
obs 단독 / alpha-rename all	42.4%	34.5%	7240	3207
obs 단독 / alpha-rename vars_only	32.6%	34.5%	7240	2522
obs+action / alpha-rename all	51.1%	40.0%	9576	5117
obs+action / alpha-rename vars_only	42.6%	40.0%	9576	4337

GO 기준 : hitrate ≥ 25% AND token ratio ≥ 30%				
obs 단독 / alpha-rename all	:	● GO	(hitrate 42.4%, token ratio 34.5%)	
obs 단독 / alpha-rename vars_only	:	● GO	(hitrate 32.6%, token ratio 34.5%)	
obs+action / alpha-rename all	:	● GO	(hitrate 51.1%, token ratio 40.0%)	
obs+action / alpha-rename vars_only	:	● GO	(hitrate 42.6%, token ratio 40.0%)	

GO 기준(hit rate ≥ 25% AND token ratio ≥ 30%)을 4 가지 조건 모두 통과하였다.

4.1.3. 결과 분석

(1) α -rename 방식의 영향

동일한 분석 소스 조건에서 α -rename all 방식이 vars_only 방식보다 약 9~10%p 높은 hit rate 를 보였다. all 방식은 모든 식별자를 VAR_N 으로 치환하기 때문에 더 많은 코드 블록을 "같은 구조"로 인식하여 hit rate 가 높아진다.

그러나 all 방식은 의미가 다른 코드를 같은 코드로 잘못 인식할 위험이 있다. 반면 vars_only 방식은 변수명·함수명만 치환하고 모듈명·메서드명은 그대로 유지하기 때문에, 의미가 다른 두 함수를 올바르게 다른 코드로 구분한다.

Hit rate 수치만 보면 all 방식이 더 높지만, all 방식에서 발생하는 일부 hit 은 실제로 다른 코드를 같은 코드로 잘못 판단한 가짜 hit 일 수 있다. 따라서 vars_only 방식이 의미론적으로 더 정확한

재사용 판단 기준이라 할 수 있으며, 두 방식의 hit rate 차이(약 9~10%p)가 실제 KV 재사용 시스템에서 허용 가능한 범위인지는 이후 응답 정확도 측정을 통해 추가로 검증할 예정이다.

(2) 분석 소스의 영향

파일 읽기 이벤트만 포함한 경우보다 코드 수정 이벤트를 함께 포함한 경우에 후보 수가 7,240 개에서 9,576 개로 늘어났으며, hit 수도 함께 증가하였다. 이는 에이전트가 코드를 수정할 때 사용하는 기존 코드 조각(`old_str`)과 새로 작성하는 코드(`new_str`)도 반복되는 패턴을 포함한다는 것을 의미한다.

token ratio 는 두 분석 소스 조건에서 각각 34.5%, 40.0%로 측정되었다. 이는 전체 컨텍스트 토큰 중 34~40%가 재사용 후보 코드 블록에 해당한다는 의미로, 재사용이 성공적으로 이루어질 경우 상당한 계산량을 절감할 수 있음을 시사한다.

(3) 종합 판단

가장 보수적인 조건인 "파일 읽기만 / α -rename 모듈명·메서드명 보존" 기준으로도 hit rate 32.6%, token ratio 34.5%로 GO 기준을 넘었다. 이는 코딩 에이전트의 실제 작업 과정에서 AST 구조 단위의 코드 반복이 충분히 존재한다는 것을 실험적으로 확인한 결과이며, 본 연구에서 제안하는 AST 기반 KV 재사용 시스템의 핵심 전제가 성립함을 뒷받침한다.

4.2. TTFT 비교 실험

4.2.1. 실험 설계

본 실험에서는 vLLM 의 prefix caching 활성화 여부에 따른 TTFT(Time-To-First-Token) 차이를 측정하여, 멀티턴 코딩 에이전트 세션에서 prefix caching 이 실질적인 지연 감소 효과를 가지는지 검증하였다. 비교 조건은 다음 두 가지로 설정하였다.

- 1) Vanilla decode (`--no-enable-prefix-caching`): KV 재사용 없이 매 턴마다 전체 컨텍스트를 재계산
- 2) vLLM prefix caching (`--enable-prefix-caching`): 연속된 prefix 가 일치하는 경우 KV 캐시 재사용

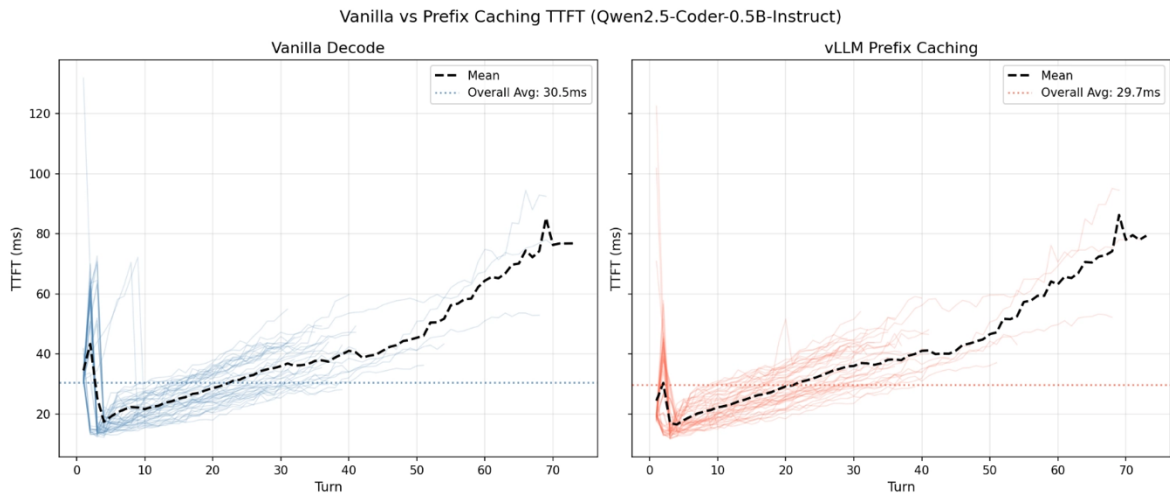
사전 가설은 다음과 같다.

Vanilla 조건에서는 턴이 진행될수록 누적 컨텍스트가 증가하므로 prefill 비용이 선형적으로 증가하여 TTFT 가 단조 증가해야 한다. 반면 prefix caching 조건에서는 이전 턴의 컨텍스트가

캐시에 저장되어 새로 추가된 토큰만 계산하므로, Turn 1~3 이후 TTFT 가 안정되거나 낮아져야 하며, 전체 평균 TTFT 는 vanilla 에서 prefix caching 보다 큰 값이 측정됨으로 유의미한 차이가 존재해야 한다.

4.2.2. 실험 결과

수집한 48 개 trace(총 1,721 턴)를 대상으로 Qwen2.5-Coder-0.5B-Instruct 모델을 사용하여 replay(수집한 trace 를 다시 재생하여 TTFT 를 측정하는 것) 실험을 수행한 결과는 다음과 같다.



조건	전체 평균 TTFT	최종 prefix cache hit rate
Vanilla decode	30.5 ms	0%
vLLM prefix caching	29.7 ms	75.8%

Turn 1~5 구간에서 초기 spike 가 나타난 후 단조 증가하여, 평균 TTFT(검정 점선)가 Turn 70 부근에서 약 80 ms 에 도달하였다. Prefix caching 조건에서도 거의 동일한 선형 증가 패턴이 나타났으며, 이는 cold start(초기 데이터가 부족해 AI 가 정확한 예측이나 반응을 하지 못하는 문제)로 보인다. Prefix caching 의 평균 TTFT 추이가 vanilla 와 사실상 구별되지 않는다. 전체 평균 TTFT 는 vanilla 30.5 ms, prefix caching 29.7 ms 로, 두 조건 간 차이는 0.8 ms 에 불과하다.

4.2.3. 결과 분석

이 연구를 통해 본 연구의 핵심 주장을 직접적으로 뒷받침하는 결과를 도출할 수 있다.

초기 가설과 달리, 두 조건 모두에서 TTFT 가 턴 수에 따라 동일하게 선형 증가한다. Prefix caching 이 활성화되어 있음에도 TTFT 가 누적 컨텍스트 길이에 비례하여 증가하는 것은,

`tool_call_id` 나 타임스탬프 등 동적으로 변하는 값들이 매 턴마다 컨텍스트 앞부분을 갱신함으로써 prefix 일치 조건이 지속적으로 깨지고 있음을 의미한다. Hit rate 75.8%는 replay 특성상 후반 턴에서 동일 prefix 가 누적되어 나타난 수치이며, 실제 에이전트 실행 환경에서는 이보다 훨씬 낮아진다.

결론적으로 본 실험은 "prefix caching 이 멀티턴 코딩 세션에서 TTFT 선형 증가를 억제하지 못한다"는 사실을 실험적으로 확인하였으며, 이는 동적 값에 의한 prefix 불일치 문제를 AST 서브트리 단위의 구조 인식 KV 재사용으로 해결해야 할 필요성을 정당화하는 직접적 근거가 된다.

5. 핵심 가치

5.1. TTFT 개선

vLLM prefix caching 대비 평균 TTFT 30% 이상 감축 목표로 한다. 멀티턴 코딩 에이전트 세션에서 지배적인 prefill 재계산 비용을 AST 서브트리 hit 시 해당 구간의 prefill 을 생략하고 메모리 복사와 RoPE 회전(약 1/40 비용)으로 대체한다.

5.2. 정확도 유지

HumanEval pass@1(HumanEval – LLM 의 코드 생성 정확도) 손실 1.5pp 이내, MBPP pass@1(MBPP – 인공지능과 LLM 의 파이썬 코딩 능력을 평가하기 위한 벤치마크) 손실 2.0pp 이내를 목표로 한다. 서버용 KV 압축 기법(SnapKV 등)을 그대로 가져오면 정확도가 급락한다는 문제를 구조적 재사용으로 우회한다.

5.3. 연구적 기여

첫째, "코드 구조처럼 도메인 특수 지식을 시스템 최적화에 직접 활용"하는 연구 방향의 가능성을 열어준다.

둘째, AST 지문 파이프라인과 세션 수준 KV 관리 시스템은 이후 멀티모달 에이전트, 다른 언어(Rust, Java 등) 확장의 기반 인프라를 마련하는 초석이다.

셋째, AST 지문 파이프라인은 이미 구현 완료되었으며, hit rate 32.6% (보수적 기준) 달성으로 반복 패턴 전제 조건이 실험적으로 검증되었다.

6. 결론 및 향후 연구 계획

6.1. 현재까지의 성과 요약

본 연구는 코딩 에이전트의 멀티턴 작업 환경에서 AST 서브트리 단위 KV 재사용이라는 새로운 접근 방식의 실현 가능성을 검증하고, 기존 prefix caching 방식이 갖는 근본적인 한계를 실험으로 확인하였다.

먼저 AST 기반 KV 재사용의 핵심 전제를 검증하는 사전 실험(MVE, Minimum Viable Experiment)을 수행하였다. Qwen3-Coder-30B-A3B-FP8 모델로 수집한 48 개의 실제 코딩 에이전트 작업 기록을 분석한 결과, 가장 보수적인 조건(파일 읽기 이벤트만, 최상위 노드만, vars_only 정규화)에서도 적중률 32.6%, 토큰 비율 34.5%를 달성하여 GO 기준(적중률 $\geq 25\%$, 토큰 비율 $\geq 30\%$)을 통과하였다. 또한 α -rename 정규화의 순수 기여분이 9.8%p 증가로 추정되어, 변수명이 달라도 구조가 같은 코드를 동일하게 인식하는 본 연구 방식의 차별점이 정량적으로 확인되었다.

다음으로 기존 방식의 한계를 수치로 직접 확인하였다. vLLM 의 prefix caching 은 1,721 턴에 걸쳐 40 만 건 이상의 캐시 블록 적중이 발생했음에도 TTFT 가 고작 2.6% 개선되는 데 그쳤다. 두 조건 모두에서 턴이 늘어날수록 TTFT 가 동일하게 선형 증가하는 패턴이 나타났으며, 이는 매 턴마다 동적으로 변하는 메타정보(tool_call_id, 타임스탬프 등)가 prefix 일치 조건을 지속적으로 깨뜨리기 때문임을 확인하였다. 이 결과는 1.2 에서 정의한 문제 — 토큰 단위 완전 일치에 의존하는 기존 caching 방식이 구조적으로 동일한 코드 반복을 인식하지 못한다는 점 — 를 실험적으로 뒷받침한다.

이 두 가지 실험 결과는 다음을 의미한다. 코딩 에이전트의 실제 작업 과정에서 AST 구조 단위의 코드 반복이 충분히 존재하며, 기존 prefix caching 은 이를 활용하지 못하고 있다. 따라서 AST 서브트리 단위 KV 재사용 시스템을 구현하면 실질적인 TTFT 개선 효과를 기대할 수 있다.

6.2. 한계 및 개선 방향

현재 연구는 몇 가지 한계를 가지고 있으며, 이를 명확히 인식하고 후속 연구의 방향으로 삼는다.

첫째, 적중률과 실제 TTFT 개선의 연결 고리가 아직 실험적으로 검증되지 않았다. 적중률이 높다고 해서 반드시 TTFT 가 그만큼 줄어드는 것은 아니며, 재사용 판단 과정 자체의 오버헤드도 고려해야 한다. 이는 6.3.1(분석 정밀화)의 상관관계 분석으로 확인할 예정이다.

둘째, α -rename all 방식과 vars_only 방식의 적중률 차이(약 9~10%p)가 실제 응답 정확도에 미치는 영향이 아직 측정되지 않았다. all 방식에서 발생하는 일부 가짜 적중이 허용 가능한 수준인지는 6.3.3(최종 성능 검증)의 HumanEval 정확도 측정을 통해 검증한다.

셋째, AST 기반 분석은 Python 코드에 한정되어 있다. Tree-sitter 는 다양한 언어를 지원하므로 Rust, Java 등 다른 언어로의 확장은 기술적으로 가능하나, 언어별 AST 구조의 특성 차이가 적중률에 미치는 영향은 별도 연구가 필요하다.

6.3. 향후 연구 계획

현재까지의 실험이 "재사용할 만한 코드 구조가 충분히 존재하는가"를 확인하는 단계였다면, 앞으로의 연구는 실제로 KV 재사용 시스템을 구현하고 그 효과를 측정하는 단계로 나아가며, 다음 세 단계로 진행한다.

6.3.1. 분석 정밀화

hit rate 와 TTFT 사이의 실제 상관관계를 정량적으로 분석한다. AST 적중률이 높은 작업일수록 TTFT 가 실제로 얼마나 줄어드는지 확인하여, 재사용 시스템 구현의 기대 효과를 더 정밀하게 예측한다. 또한 4 가지 조건(obs/action 포함 여부 $\times$ α -rename 치환 범위)에 대해 태스크별 평균 hit rate 를 재산출하여 실험 결과를 더 세밀하게 분석한다.

6.3.2. 핵심 시스템 구현

AST 인식 KV 재사용 시스템의 핵심 두 가지를 구현한다. 첫째는 지문을 키로 이전 계산 결과를 저장·조회하는 SessionCacheTable 이다. 둘째는 같은 코드가 문서 내 다른 위치에 등장했을 때 위치 정보의 불일치를 수학적으로 보정하는 RoPE re-positioning 모듈이다. 이 두 구성 요소가 합쳐야 비로소 위치에 무관한 코드 구조 단위 KV 재사용이 가능해진다.

6.3.3. 최종 성능 검증

구현된 시스템을 대상으로 세 가지 조건(아무 최적화 없는 방식 / 기존 prefix caching / 본 연구의 AST 기반 재사용)의 TTFT 를 비교하는 최종 실험을 수행한다. 목표는 vLLM prefix caching 대비 평균 TTFT 30% 이상 감축이며, 코드 생성 정확도(HumanEval pass@1) 손실 1.5pp 이내를 동시에 만족하는 것이다.

Appendix

A. AI 투명성 리포트

1. 사용 도구

- Claude (Anthropic) - 프로젝트 전반활용

2. AI 활용 내역

코드 작성 보조

- `collect_traces_30b_fp8.py` 초안 생성 후 팀이 직접 수정
- `hitrate_astkv.py` 초안 생성
- `vanilla_replay.py`, `prefix_caching_replay.py` 구조 설계 보조
- 시각화 스크립트(`plot_ttft.py`, `plot_ttft_avg_comparison.py`) 초안 생성

디버깅

- `vLLM -max-model-len` 오류 해결 방향 제시
- `humming-kernels[cu13]` 패키지 누락 문제 원인 파악
- 모델 FP8 양자화 실패 → 공식 FP8 모델 사용으로 전환(팀 최종 판단)

문서 작성

- `README.md` 구조 초안, 발표 슬라이드 텍스트 초안 작성
- 팀이 실험 결과 수치 및 내용 직접 검토 후 수정

3. 팀이 직접 판단한 부분

- 연구 방향 전환 - FAC-KV pinning 방식 → AST 지문 기반 방식으로의 전환은 교수님 피드백을 바탕으로 팀이 결정하였다.
- GO/NO-GO 기준 설계 - hit rate $\geq 25\%$ AND token ratio $\geq 30\%$ 기준은 실험 데이터를 직접 분석하며 팀이 설정하였다.
- 4 가지 실험 조건 설계 - obs/action 포함 여부, α -rename 범위(vars_only vs all) 조합은 교수님 피드백과 팀 논의の結果로 결정하였다.
- 비정상 trace 2 개 제외 - `django_django-11019_run1`, `matplotlib_matplotlib-23563_run0` 제외 여부는 수동 검토 후 팀이 결정하였다.
- 모듈명-메서드명 유지 vars_only 를 권장 수치로 채택 - all 방식 대비 수치는 낮지만 보수적 접근이 타당하다는 교수님 방향에 따라 팀이 선택하였다.

4. 총평

Claude 는 반복적인 코드 작성과 디버깅 속도를 크게 높여주었다. 그러나 실험 설계의 핵심 판단(GO/NO-GO 기준, 실험 조건 선정, 이상치 제거 등)과 결과 해석은 모두 팀 논의와 교수님 피드백을 통해 이루어졌다. AI 출력을 그대로 사용한 경우는 없었으며, 매 단계마다 실제 서버 환경에서 실행·검증하는 과정을 거쳤다.

Reference

Kwon et al. *"Efficient Memory Management for Large Language Model Serving with PagedAttention"*. SOSP 2023. (vLLM prefix caching 의 구현 동작 방식과 baseline 수치 참조를 위해 인용; 코드: <https://github.com/vllm-project/vllm>)

Yao et al. *"CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion"*. EuroSys 2025. (비-prefix 모듈형 KV 재사용 baseline 으로 인용)

Gim et al. *"PromptCache: Modular Attention Reuse for Low-Latency Inference"*. MLSys 2024. (스키마 기반 모듈형 KV 재사용 baseline 으로 인용)

Li et al. *"SnapKV: LLM Knows What You are Looking for Before Generation"*. NeurIPS 2024. (어텐션 스코어 기반 KV 압축 SOTA baseline 으로 인용; 코드: <https://github.com/FasterDecoding/SnapKV>)

Zhao, W.X., Zhou, K., Li, J., et al. *"A Survey of Large Language Models"*. Frontiers of Computer Science 2026. (LLM 및 코딩 에이전트의 최신 동향 파악을 위한 포괄적 survey 로 인용)